
PROYECTO KONTROL · SPRINT 7

Inventario de deuda técnica y plan de refactorización — Sprint 8

Campo	Valor
Documento	Inventario de deuda técnica del proyecto Kontrol
Versión	1.0
Fecha	07/09/2026
Rama analizada	deve`lop @ 264ec19
Sprint de elaboración	Sprint 7
Sprint de ejecución	Sprint 8

1. Método y fuentes de evidencia

Cada elemento del inventario sale de una medición reproducible sobre el repositorio, no de percepción. Los comandos usados:

Fuente	Comando / artefacto	Resultado
Cobertura backend	<code>npm run test:coverage -w backend</code>	37.05 % stmts / 23.96 % branches / 27.44 % funcs / 37.24 % lines sobre 1 719 sentencias medidas
Cobertura frontend	<code>npm run test:coverage -w frontend</code>	91.92 / 86.49 / 90.51 / 93.08 sobre 929 sentencias medidas
Archivos fuera del denominador	<code>diff de coverage-summary.json contra find src -name '*.js'</code>	backend: 59 de 94 archivos sin medir; frontend: 23 de 134 medidos
Dependencias	<code>npm audit --json</code>	17 vulnerabilidades : 2 críticas, 8 altas, 6 moderadas, 1 baja
Rendimiento	<code>docs/pruebas-carga-estres.md</code> (SCRUM-28)	4 de 6 casos de carga fallan p95; E1 y E5 degradan en su carga base
Interfaz	Observación directa + auditoría de estilos (SCRUM-27)	fondo animado sin scrim, tema claro sin paleta, 38 <code>!important</code> , 27 valores distintos de <code>z-index</code>
Duplicación	<code>grep</code> de helpers repetidos	14 definiciones locales de <code>authHeader/authHeaders</code>
Peso del bundle	<code>npm run build -w frontend</code>	<code>chunk</code> de entrada de 1 236 kB (372 kB comprimidos) con aviso de Vite; <code>dist/assets</code> de 8.3 MB
Seguridad	Revisión manual de la pila de middlewares, las salidas HTTP, el aislamiento del agente y la configuración del pool	Sin cabeceras de seguridad, sin límite de intentos, SSRF en el probador de integraciones, <code>fetch</code> sin timeout, TLS de base de datos sin verificar
Deuda ya registrada	<code>docs/plan-maestro-pruebas.md</code> , caso SEC2 y riesgo 8	Hallazgo de cabeceras ausentes aceptado el 04/09/2026 con la corrección aplazada a una tarea que nunca se creó

2. Tabla de deuda técnica priorizada

Prioridad: **P1** = riesgo de corrupción de datos, caída del servicio o pérdida de dinero · **P2** = fricción de mantenimiento o de usuario medible · **P3** = mejora estructural sin impacto inmediato.

Esfuerzo en story points (1 SP ≈ 2 h de trabajo efectivo del equipo), acotado al alcance del Sprint 8.

ID	Área	Descripción	Causa de origen	Impacto	Esf.	Prio.	Técnica de refactorización propuesta	Sprint
DT-01	Backend / robustez	No existe middleware de errores ni envoltorio <code>asyncHandler</code> . 19 <code>throw err</code> en 8 controladores viajan como <i>promise rejection</i> no capturada; Express 4 no las intercepta y Node 20+ termina el proceso	El proyecto creció endpoint a endpoint sin definir nunca la capa de errores transversal	Una violación de FK o un fallo de pool en cualquier endpoint tumba el backend entero para todos los usuarios	3	P1	Extracción de método (<code>asyncHandler</code>) + middleware central de errores, con pruebas de caracterización del formato de error actual	8
DT-02	POS / dinero	El backend acepta <code>precio_unitario</code> tal como lo manda el cliente y nunca lo contrasta con <code>producto.precio_venta</code> . El descuento y el IVA se calculan solo en el navegador (<code>calcSale</code>) y no se persisten ni se envían	El POS se construyó de frontend hacia atrás; el endpoint de venta se limitó a registrar movimientos	Un cliente manipulado registra ventas a precio arbitrario; el informe de ventas y el ticket del POS divergen en cuanto haya descuento o IVA	5	P1	Unificación de capa de servicios (mover el cálculo a un módulo compartido) + pruebas de caracterización sobre el total actual antes de mover nada	8
DT-03	Inventario / lógica	La unicidad de <code>codigo_barras</code> es por proyecto (índice parcial sobre (<code>id_proyecto</code> , <code>codigo_barras</code>)), no por empresa; y <code>handleScan</code> resuelve con <code>products.find(...)</code> , es decir el primer match	La restricción se añadió en SCRUM-18 pensando en el alta de producto, no en la resolución del escaneo en la vista «todos los proyectos»	Con el mismo código en dos proyectos, el escáner mete al carrito el producto equivocado y descuenta stock del que no era, sin ningún aviso	3	P1	Pruebas de caracterización del escaneo + extracción de método (<code>resolveScannedProduct</code>) + corrección de datos y ajuste de la restricción	8
DT-04	Backend / defecto	Desajuste de parámetro de ruta: <code>productRoutes.js</code> declara <code>/:id/suppliers/:supplierId</code> , y <code>updateSupplierLink/unlinkSupplier</code> leen <code>req.params.pid</code> (siempre <code>undefined</code>)	Se renombró el parámetro en la ruta y en el esquema, y no en el controlador; ningún test cubre esos endpoints	Los tres endpoints de producto-proveedor están rotos en producción: el <code>UPDATE/DELETE</code> no encuentra fila o revienta la consulta	1	P1	Corrección puntual + prueba de caracterización que fije el contrato del parámetro	8
DT-05	Autorización	Coexisten tres modelos: middleware de ruta (<code>requireProjectPermission</code> en <code>POST /products</code>), guarda dentro del controlador (<code>ensureProductInventoryAccess</code> en <code>PUT/DELETE</code>) y rol de empresa (<code>requireCompanyRole</code> en <code>/suppliers</code>). El mismo recurso se protege de forma distinta según el verbo	Cada módulo resolvió la autorización por su cuenta; no hay una regla escrita ni un test que la fije	Precedente ya vivido: los métodos de editar y borrar producto eran inalcanzables desde el frontend. Quedan huecos equivalentes sin detectar en los módulos sin cobertura	5	P1	Pruebas de caracterización por endpoint + unificación de capa (un único <code>requireProjectPermission</code> de ruta). En el Sprint 8 se acota a inventario y proveedores; el resto de módulos queda en backlog	8

ID	Área	Descripción	Causa de origen	Impacto	Esf.	Prio.	Técnica de refactorización propuesta	Sprint
DT-06	Pruebas / CI	El denominador de cobertura solo incluye lo que algún test importa (<code>coverage.include</code> sin declarar). Backend: 1 719 de ~14 900 líneas medidas, 59 de 94 archivos invisibles. Frontend: 23 de 134 archivos. Tres umbrales por módulo (<code>ReportsView.vue</code> , <code>ReportDetailView.vue</code> , <code>components/reports/**</code>) casan con cero archivos y pasan en vacío	Decisión consciente de SCRUM-23 para que el trinquete no dejara el pipeline en rojo; el comentario del propio config lo documenta	El gate da 37 % y 92 % como si fueran cobertura real. Un módulo entero puede entrar sin una sola prueba sin mover el número. Es además prerrequisito de DT-02, DT-03 y DT-05: sin denominador real no hay red de seguridad para tocarlos	5	P2	Declarar <code>coverage.include</code> , remedir la línea base real y reajustar el trinquete	8
DT-07	Frontend / arquitectura	No hay capa de acceso a API. 14 definiciones locales de <code>authHeader/authHeaders</code> , <code>fetch</code> crudo en 24 archivos, <code>localStorage.getItem('token')</code> leído en 10 sitios y dos formas distintas de resolver la URL base	Las vistas se escribieron antes que <code>services/</code> , y <code>services/</code> solo cubre 4 dominios (agent, auth, integrations, marketing)	Cualquier cambio transversal —refresh de token, 401 global, reintentos, cabecera de empresa— hay que replicarlo en 24 sitios. Es el multiplicador de coste de casi todo el resto	5	P2	Unificación de capa de servicios (<code>services/http.js</code> + un servicio por dominio). En el Sprint 8 se crea la capa y se migran Inventario y Dashboard	8
DT-08	Dependencias	17 vulnerabilidades: críticas <code>shell-quote</code> (vía <code>concurrently</code>) y su cadena; altas <code>ws</code> , <code>vite</code> , <code>nanoid</code> , <code>brace-expansion</code> , <code>socket.io-parser</code> , <code>uploadthing/effect</code> ; moderadas <code>dompurify</code> (10 bypasses de saneado), <code>mongoose</code> (prototype pollution), <code>qs</code> , <code>engine.io</code>	No hay revisión periódica de dependencias ni step de auditoría en CI	XSS real vía <code>dompurify</code> (se usa para renderizar markdown del agente), y <code>uploadthing</code> solo se arregla con un cambio semver-major	3	P2	Actualización de dependencias por lotes (parches → menores → <code>uploadthing</code> mayor aparte) + <code>npm audit</code> en el pipeline	8
DT-09	Interfaz / estilos	Fondo <code>Waves</code> global (<code>App.vue</code>) pintado con <code>color1.3 = var(--Text)</code> , el mismo token del texto, sin capa de contraste sobre el contenido. Tema claro declarado pero vacío (<code>theme.css:249</code> , TODO SCRUM-12): hereda el oscuro. 58 archivos aún con variables <code>legacy --Primary/--Text</code> , hasta 83 literales hexadecimales por archivo, 38 <code>!important</code> y 27 valores distintos de <code>z-index</code> entre <code>-1</code> y <code>9999</code>	Las fases 1 y 2 del rediseño migraron parte de las pantallas; el resto quedó congelado con alias <code>legacy</code> , y el fondo animado se añadió antes que los tokens	Texto ilegible cuando la onda pasa por detrás (lo reportado); <code>prefers-color-scheme: light</code> entrega una pantalla oscura; superposiciones impredecibles de modales y overlays	5	P2	Extracción de componente (<code><AppSurface></code> con <code>scrim</code>) + escala única de <code>z-index</code> en <code>theme.css</code> . La retirada de alias <code>legacy</code> y literales hex queda en backlog	8
DT-10	Rendimiento	<code>loadOverview</code> en <code>DashboardView.vue</code> hace 1 petición de proyectos + N peticiones a <code>/api/budgets/project/:id/summary</code> (hasta 100 en paralelo)	Se resolvió en el cliente lo que faltaba como endpoint agregado en el backend	Es el patrón que SCRUM-28 identificó (hallazgo 3): las peticiones con varias consultas por acción son las que menos margen tienen antes de degradar	3	P2	Extracción a un endpoint agregado (<code>GET /api/budgets/summary?projectIds=</code>) + caché corta de agregaciones	8

ID	Área	Descripción	Causa de origen	Impacto	Esf.	Prio.	Técnica de refactorización propuesta	Sprint
DT-11	Calidad / CI	Sin ESLint, sin Prettier, sin <code>.editorconfig</code> . El pipeline solo corre tests y build: no hay lint ni auditoría de dependencias. 71 llamadas a <code>console.*</code> en código de producción y 13 marcas TODO/FIXME, 3 de ellas sin ticket asociado	Nunca se configuró; el estilo se sostiene por revisión manual en PR	Defectos triviales (DT-04 es literalmente uno) llegan a <code>develop</code> sin que nada los detecte; ruido en logs de producción	3	P3	Actualización de herramientas: ESLint + Prettier con la configuración de Vue 3, más lint y npm audit --audit-level=high como steps de CI	8
DT-12	Seguridad	El probador de integraciones (POST /api/integrations/:slug/test) hace fetch a una URL que el propio usuario guarda en <code>credentials</code> , sin lista blanca ni bloqueo de direcciones internas. <code>saveIntegrationSchema</code> declara <code>credentials: z.record(z.string(), z.string())</code> : el campo <code>url</code> no se valida en absoluto	El probador se escribió para dar realimentación inmediata al configurar un webhook; nunca se planteó que la URL la elige el usuario	SSRF. Se exige rol owner o admin de empresa, pero cualquier usuario registrado crea su propia empresa con POST /api/companies y es su owner. Desde ahí alcanza servicios HTTP internos de la VM que no están expuestos a internet, y el status de la respuesta se le devuelve como oráculo	3	P1	Extracción de método (<code>assertPublicHttpUrl</code>): validar esquema, resolver el host y rechazar rangos privados, loopback y enlace local, antes de cualquier salida HTTP	8
DT-13	Seguridad / rendimiento	No hay límite de intentos en POST /api/auth/login ni en ningún otro endpoint. No se usa <code>express-rate-limit</code> ni equivalente	Nunca se configuró; el proyecto no tiene capa de middlewares transversales (misma raíz que DT-01)	Fuerza bruta contra contraseñas sin freno. Y peor: SCRUM-28 midió que el login es el endpoint más caro en CPU por <code>bcrypt.compare</code> , y que ya degrada en su carga base (hallazgo 5, caso E1). Un atacante no autenticado tumba el servicio con peticiones de login	3	P1	Extracción de método (middleware de límite por IP y por cuenta) aplicado primero a /auth, después a los endpoints caros	8
DT-14	Seguridad	El backend no emite ninguna cabecera de seguridad HTTP: sin política de seguridad de contenido, sin <code>X-Content-Type-Options</code> , sin protección de clickjacking, sin HSTS ni control de <code>Referrer</code> . La pila de middlewares es solo <code>cors</code> y <code>express.json</code>	El plan maestro de pruebas ya registró el hallazgo el 04/09/2026 (caso SEC2, riesgo 8) y dejó la corrección explícitamente fuera del alcance de HU-38, como «una tarea de desarrollo». Esa tarea nunca se creó	El caso SEC2 de la matriz de trazabilidad sigue en estado Pendiente y no puede pasar a Cubierto. Sin política de seguridad de contenido, las 10 vulnerabilidades de saneado de <code>dompurify</code> (DT-08) pierden su última barrera, y el token vive en <code>localStorage</code> , accesible desde cualquier script inyectado	2	P2	Actualización de herramientas: añadir <code>helmet</code> a la pila de middlewares y fijar las cinco cabeceras que el plan maestro exige verificar	8
DT-15	Backend / resiliencia	Ninguna llamada HTTP saliente lleva <code>timeout</code> : <code>webhook</code> , Slack, Teams, Telegram, Twilio y SendGrid usan <code>fetch</code> sin <code>AbortSignal</code> . El pool de Postgres se crea sin <code>max</code> , sin <code>connectionTimeoutMillis</code> ni <code>idleTimeoutMillis</code> , y con <code>ssl: { rejectUnauthorized: false }</code> cuando <code>DATABASE_SSL</code> está activo	Cada integración se añadió copiando el <code>fetch</code> de la anterior; el pool se escribió al principio del proyecto y no se revisó	Una integración que no responde retiene conexiones de forma indefinida. El pool usa el máximo por defecto de 10, valor que nadie eligió y que condiciona directamente los cuellos de botella medidos en SCRUM-28. Y <code>rejectUnauthorized: false</code> acepta cualquier certificado en la conexión a la base de datos, lo que anula el valor de TLS frente a un intermediario	3	P2	Extracción de método (un <code>fetchWithTimeout</code> compartido por los seis servicios) + parametrizar el pool y verificar el certificado	8

ID	Área	Descripción	Causa de origen	Impacto	Esf.	Prio.	Técnica de refactorización propuesta	Sprint
DT-16	Agente de IA	<code>executeReadOnlY</code> llama a <code>setupAgentScope</code> antes de abrir la transacción: las ~20 sentencias <code>CREATE OR REPLACE TEMP VIEW</code> se ejecutan en <code>autocommit</code> , así que las vistas sobreviven a la transacción, quedan en la conexión al devolverla al pool y nunca se eliminan. Llevan el nombre de las tablas reales (<code>usuario</code> , <code>proyecto</code> , <code>producto</code> , <code>empresa</code>) y filtran por la empresa de quien preguntó	Las vistas temporales se añadieron para acotar al agente por empresa y proyecto; el orden respecto al <code>BEGIN</code> no se revisó porque en un <code>BEGIN TRANSACTION READ ONLY</code> no se pueden crear	Hoy no hay fuga porque todas las consultas de la aplicación cualifican con <code>public.</code> , comprobado sobre <code>backend/src</code> . Es una trampa latente: la primera consulta sin cualificar que alguien escriba resolverá contra la vista temporal de otro inquilino, porque <code>pg_temp</code> se busca antes que <code>public</code>	3	P2	Extracción de método (<code>withAgentScope</code>): crear las vistas dentro de la transacción y eliminarlas al terminar, o aislar al agente en su propio pool con un rol de solo lectura, que es la capa 3 de defensa que el propio archivo documenta y que no está implementada	8
DT-17	Frontend / rendimiento	El bundle de entrada pesa 1 236 kB (372 kB comprimidos) en un solo <i>chunk</i> , y Vite avisa de ello en cada build. La causa es <code>KontrolLogo3D</code> , que importa <code>three</code> entero y lo usan <code>LoginView</code> y <code>RegisterView</code> . A eso se suma <code>ogl</code> en el fondo global de <code>App.vue</code> , más <code>gsap</code> y <code>lenis</code> : cuatro librerías de animación en el arranque. <code>dist/assets</code> ocupa 8.3 MB	El logo 3D y los fondos animados se añadieron por identidad visual, sin medir el coste sobre la primera pantalla	Un usuario que solo va a iniciar sesión descarga 372 kB comprimidos de motor 3D antes de ver el formulario. Es la misma pantalla que SCRUM-28 señala como la más lenta del sistema por el lado del servidor	5	P2	Extracción de componente con carga diferida (<code>defineAsyncComponent</code> para el logo y los fondos) + <code>manualChunks</code> para separar las librerías de animación, con una alternativa estática mientras el 3D carga	8

Totales: 17 elementos · 7 en P1 · 9 en P2 · 1 en P3 · 60 SP.

3. Detalle de la evidencia por elemento

Referencias exactas para quien ejecute la refactorización.

DT-01 — Errores asíncronos sin capturar

- `backend/src/index.js:30-38` — la cadena es `cors` → `express.json` → `/api`. No hay `app.use((err, req, res, next) => ...)`.
- `backend/src/controllers/productController.js:303` — `throw err` dentro de un handler `async`.
- Mismo patrón en `companyController` (4), `teamController` (4), `productController` (3), `inventoryMovementController` (2), `projectController` (2), `taskController` (2), `budgetController` (1), `supplierController` (1).
- No existe `asyncHandler` ni `next(err)` en todo `backend/src`.

DT-02 — El cliente fija el precio de venta

- `backend/src/schemas/inventoryMovementSchemas.js` — `createSaleSchema` acepta `precio_unitario: z.number().min(0).optional().default(0)`.
- `backend/src/controllers/inventoryMovementController.js:389` — inserta `item.precio_unitario ?? 0` sin leer el precio del producto, pese a que la fila ya está bloqueada con `FOR UPDATE` unas líneas antes.
- `frontend/src/views/InventoryPage.vue:900` — el navegador envía `Number(item.product.precio_venta)`.
- `frontend/src/utils/sales.js:108` — `calcSale` (descuento + IVA 12 %) **no lo importa nadie**; solo `lineTotal` y `calcSubtotal` están en uso. Su test `frontend/tests/sales.test.js` cubre lógica muerta y cuenta para el umbral de cobertura.

DT-03 — Código de barras duplicado

- `backend/src/db/bootstrap.js:238-240` — `CREATE UNIQUE INDEX ... ON producto (id_proyecto, codigo_barras) WHERE codigo_barras IS NOT NULL`: el ámbito es el proyecto.
- `frontend/src/views/InventoryPage.vue:780` — `products.value.find(p => String(p.codigo_barras) === scanned)`: primer match, sin comprobar ambigüedad, y `products` puede abarcar varios proyectos.
- El esquema se aplica con `ensureDatabaseSchema()` en el arranque, sin migraciones versionadas: endurecer el índice sobre datos sucios haría fallar el arranque del backend. La corrección de datos tiene que preceder al cambio de índice.

DT-04 — Parámetro de ruta desajustado

- `backend/src/routes/productRoutes.js:81,87` — `('/:id/suppliers/:supplierId')`.
- `backend/src/schemas/productSchemas.js` — `productSupplierParamsSchema` valida `supplierId`.
- `backend/src/controllers/productController.js:367,384,395,407` — lee y usa `pid`.

DT-05 — Autorización dispersa

- `backend/src/routes/productRoutes.js:52-57` — `POST /` protege con `requireProject` + `requireProjectPermission('gestionar_inventario')`.
- `backend/src/routes/productRoutes.js:60-70` — `PUT /:id` y `DELETE /:id` sin middleware de permiso; la comprobación vive dentro del controlador (`ensureProductInventoryAccess`).
- `backend/src/routes/supplierRoutes.js:36-53` — tercer modelo, `requireCompanyRole('owner', 'admin', 'manager')`.
- Los módulos sin cobertura de DT-06 incluyen `productController`, `inventoryMovementController`, `companyController`, `teamController` y `authController`.

DT-06 — Cobertura sobre denominador parcial

- `backend/vitest.config.js` y `frontend/vitest.config.js` documentan la decisión en comentario: «Sin declarar `coverage.include`, solo se mide lo que algún test importa; un archivo que nadie prueba no aparece en el reporte y no baja el porcentaje. Es intencional».

- Los propios configs avisan: «un glob que no casa con ningún archivo del reporte pasa en vacío, sin avisar». Es exactamente lo que ocurre hoy con los tres umbrales del módulo de reportes en el frontend.
- Los umbrales de `frontend/vitest.config.js` (83/83/78/85) están calibrados sobre una base de 168 sentencias en 6 archivos; hoy el reporte mide 929 sentencias en 23 archivos y da 91.92 %. El trinquete quedó desfasado hacia abajo.

DT-07 — Sin capa de acceso a API

- 14 definiciones locales de `authHeader` / `authHeaders`, entre ellas `DashboardView.vue:315`, `InventoryPage.vue:506`, `ProjectsView.vue:545`, `ReportsView.vue:311`, `ReportDetailView.vue:315`, `BudgetView.vue:42`, `ProjectDetailView.vue:613`, `ProductDetailView.vue:225`.
- `fetch` crudo en 24 archivos; los que más concentran: `DashboardView.vue` (11), `ProjectDetailView.vue` (8), `InventoryPage.vue` (6), `ProjectMembersPanel.vue` (6).
- Dos resoluciones distintas de la URL base: `stores/chat.js:13` y `TaskDetailModal.vue:86` usan `import.meta.env.VITE_API_URL || window.location.origin`; el resto asume rutas relativas `/api/...`.

DT-09 — Contraste y estilos

- `frontend/src/App.vue:4-19` — `LineWaves` con `color1/2/3="var(--Text)"` y `brightness=0.08`, dentro de `.background { position: fixed; inset: 0; z-index: 0 }`; el contenido va en `z-index: 1` sin ninguna superficie opaca ni scrim intermedio. El texto claro cae sobre líneas del mismo token de color.
- `frontend/src/styles/theme.css:249-267` — `[data-theme='light'] { /* overrides del tema claro van aquí */ }` está vacío a propósito.
- Archivos con más literales hexadecimales: `ChatView.css` (83), `ReportDetailView.vue` (79), `FloatingChat.vue` (76), `AgentView.vue` (71).
- `!important: 13` en `LoginView.css`, 13 en `RegisterView.css`, 2 en `Waves.vue`.
- `prefers-reduced-motion` solo se respeta en 3 de los fondos animados.

DT-10 — N+1 en el dashboard

- `frontend/src/views/DashboardView.vue:253-266` — `Promise.allSettled` sobre un `map` de los proyectos.
- Alineado con `docs/pruebas-carga-estres.md`, sección 4, hallazgo 3: «C2 y C5 hacen más de una consulta por petición de usuario ... son los que menos margen tienen antes de degradar», y hallazgo 5: «E1 y E5 degradan en su propio nivel base» (login y venta del POS).

DT-12 — SSRF en el probador de integraciones

- `backend/src/services/webhookService.js:18` y `:45` — `fetch(url, ...)` con la URL que llega de las credenciales guardadas, sin validación previa.
- `backend/src/schemas/integrationSchemas.js:3-6` — `saveIntegrationSchema` acepta `credentials: z.record(z.string(), z.string())`: ninguna clave se valida, `url` incluida.
- `backend/src/routes/integrationRoutes.js:25` — `POST /:slug/test` exige `requireCompanyRole('owner', 'admin')`.

- `backend/src/routes/companyRoutes.js:44` — `POST /companies` solo exige `requireAuth`: quien se registra crea empresa y queda como su owner, de modo que la barrera de rol no filtra a nadie.
- El despliegue corre en una VM de Azure con Caddy y varios contenedores en la misma red (`docker-compose.prod.yml` , `Caddyfile`), todos alcanzables por HTTP desde el backend.

DT-13 — Sin límite de intentos

- `backend/package.json` — no hay `express-rate-limit` ni ningún paquete equivalente entre las dependencias.
- `backend/src/index.js:30-38` — la pila de middlewares es `cors` , `express.json` y el router.
- `backend/src/controllers/authController.js:14` — `SALT_ROUNDS = 10` en cada `bcrypt.compare` .
- `docs/pruebas-carga-estres.md` , sección 4, hallazgo 5: «E1 y E5 degradan en su propio nivel base, sin necesitar ningún escalón de estrés adicional». E1 es precisamente el login.

DT-14 — Cabeceras de seguridad ausentes

- `docs/plan-maestro-pruebas.md` , sección 9.2: «el backend hoy solo aplica los middlewares `cors` y `express.json` , y no agrega ninguna de estas cabeceras de seguridad. Este plan registra el hallazgo; la corrección es una tarea de desarrollo fuera del alcance de HU-38».
- Misma fuente, riesgo 8: «Alta probabilidad de persistir, medio impacto».
- Misma fuente, matriz de trazabilidad, caso SEC2: estado **Pendiente**.
- El repositorio no contiene ninguna tarea ni rama que recoja esa corrección: es deuda aceptada y sin dueño desde el 04/09/2026.

DT-15 — Salidas HTTP y pool sin endurecer

- `backend/src/services/webhookService.js` , `slackService.js` , `teamsService.js` , `telegramService.js` , `twilioSmsService.js` , `sendgridService.js` — ningún `fetch` pasa `signal` ; `AbortSignal` no aparece en ninguno de los seis.
- `backend/src/db/pool.js:5-8` — el `Pool` recibe solo `connectionString` y `ssl` ; `max` queda en el valor por defecto de `pg` , que es 10.
- `backend/src/db/pool.js:7` — `ssl: process.env.DATABASE_SSL === 'true' ? { rejectUnauthorized: false } : false` .

DT-16 — Vistas temporales del agente fuera de la transacción

- `backend/src/services/agentSql.js:364-377` — el orden es `pool.connect()` → `setupAgentScope(client, scope)` → `BEGIN TRANSACTION READ ONLY` → `SET LOCAL search_path = pg_temp` .
- `backend/src/services/agentSql.js:64` — `CREATE OR REPLACE TEMP VIEW ${name} AS ${selectSql}` ; las vistas duran lo que la sesión, no la transacción, y no hay ningún `DROP` .
- `backend/src/services/agentSql.js:76-200` — se crean con los nombres `rol` , `permiso_empresa` , `empresa` , `empresa_usuario` , `categoria` , `proyecto` , `usuario` , `producto` , `proveedor` y otras, filtradas por `id_empresa` e `id_proyecto` de quien pregunta.

- Comprobación de que hoy no hay fuga: `grep -E "(FROM|JOIN)\s+(public\.)?[a-z_]+"` sobre `backend/src` no devuelve ninguna tabla real sin cualificar; los únicos casos sin `public.` son nombres de CTE (`movs` , `through` , `activity_totals`) y catálogos del sistema en `bootstrap.js` .
- `backend/src/services/agentSql.js:9-10` — el propio archivo declara la capa 3 de su defensa en profundidad: «The connecting role itself should ideally be granted SELECT only». No está implementada: el agente usa el mismo pool y el mismo rol que el resto de la aplicación.

DT-17 — Peso del bundle de entrada

- Salida de `npm run build -w frontend: index-CVpLTA07.js` con 1 236.10 kB (371.76 kB comprimidos) y `index-Bco10w8o.js` con 459.11 kB (120.13 kB comprimidos), más el aviso de Vite «Some chunks are larger than 500 kB after minification».
- `frontend/src/components/UI/KontrolLogo3D/KontrolLogo3D.vue:12` — `import * as THREE from 'three'` , sin importación selectiva.
- Lo consumen `LoginView.vue:10` , `RegisterView.vue:128` y `LandingPage.vue:86` , es decir las tres pantallas públicas.
- `frontend/src/App.vue:34` — `Waves (ogl)` se importa de forma estática y se monta en todas las rutas.
- `frontend/dist/assets` ocupa 8.3 MB en 72 archivos.

4. Técnicas de refactorización propuestas, y por qué cada una

Técnica	Elementos	Justificación
Pruebas de caracterización antes de refactorizar	DT-01, DT-02, DT-03, DT-04, DT-05	Son los módulos con 0 % de cobertura real. Fijar el comportamiento actual —incluido el incorrecto— es condición previa para poder cambiarlo y saber qué se rompió. Es la misma disciplina «plan antes que prueba» del plan maestro.
Extracción de método	DT-01, DT-03, DT-10, DT-12, DT-13, DT-15, DT-16	Lógica atrapada dentro de un handler o de una vista que hay que sacar a una función pura para poder probarla. Precedente ya aplicado en el repositorio: <code>frontend/src/utills/sales.js</code> . Los cuatro últimos son el mismo patrón aplicado a un control transversal que hoy no existe en ningún sitio: validar una URL de salida, limitar intentos, poner timeout a un <code>fetch</code> y acotar el alcance del agente.
Extracción de componente	DT-09, DT-17	La superficie con <code>scrim</code> se extrae una vez y la consumen todas las vistas, en lugar de parchear el contraste pantalla a pantalla. En DT-17 la extracción es además diferida: el logo 3D deja de formar parte del bundle de entrada.
Unificación de capa de servicios	DT-02, DT-05, DT-07	Tres capas hoy fragmentadas: acceso HTTP en el frontend, autorización en el backend y cálculo de la venta repartido entre ambos. Una fuente de verdad por capa.

Técnica	Elementos	Justificación
Actualización de dependencias y herramientas	DT-08, DT-11, DT-14	Vulnerabilidades con parche disponible, ausencia de linter y ausencia de cabeceras de seguridad. Los tres se resuelven instalando y configurando, sin tocar lógica de negocio, por eso pueden entrar en paralelo al resto.

5. Propuesta de tareas

Desglose de cada elemento de la sección 2 en tareas ejecutables, derivadas de la técnica propuesta para ese elemento.

Dos reglas de orden que atraviesan toda la lista:

- T-11 va primero.** Mientras el denominador de cobertura no incluya los archivos que se van a tocar (DT-06), ninguna prueba de caracterización protege nada: el gate seguiría en verde aunque se borrara.
- La caracterización precede a su refactorización.** Las tareas de tipo "caracterizar" fijan el comportamiento actual incluido el incorrecto y se mergean antes que la tarea que lo cambia. Es la técnica propuesta para DT-01, DT-02, DT-03, DT-04 y DT-05.

Tarea	Título	DT	Técnica que aplica	Entregable	SP	Prio.	Depende de
T-01	Caracterizar el contrato de error actual de la API	DT-01	Pruebas de caracterización	Suite que fija status y cuerpo de las respuestas de error que hoy devuelven los controladores	2	P1	—
T-02	Extraer <code>asyncHandler</code> y añadir middleware central de errores	DT-01	Extracción de método	<code>asyncHandler</code> aplicado a los 8 controladores + <code>app.use((err, req, res, next) => ...)</code> en <code>index.js</code> ; ningún <code>throw</code> escapa del proceso	3	P1	T-01
T-03	Caracterizar el total de venta actual	DT-02	Pruebas de caracterización	Test sobre <code>createSale</code> que documenta que hoy se persiste el precio del cliente, sin descuento ni IVA	2	P1	T-11
T-04	Unificar el cálculo de la venta en un módulo compartido	DT-02	Unificación de capa de servicios	Módulo único de totales consumido por el POS y por el backend; el servidor lee <code>precio_venta</code> de la base, recalcula descuento e IVA y los persiste	5	P1	T-03
T-05	Caracterizar la resolución del escaneo de código de barras	DT-03	Pruebas de caracterización	Test con dos productos que comparten código en proyectos distintos, que deja constancia de que hoy gana el primero	2	P1	T-11
T-06	Extraer <code>resolveScannedProduct</code> y rechazar el escaneo ambiguo	DT-03	Extracción de método	Función pura que devuelve match único, «no encontrado» o «ambiguo»; la vista muestra el tercer caso en vez de elegir por su cuenta	3	P1	T-05
T-07	Corregir códigos duplicados y elevar la unicidad a empresa	DT-03	Corrección de datos + ajuste de restricción	Script que detecta y resuelve duplicados, ejecutado antes de sustituir el índice por <code>(id_empresa, codigo_barras)</code>	3	P1	T-06
T-08	Corregir el parámetro <code>supplierId</code> en los endpoints de producto y proveedor	DT-04	Corrección puntual + caracterización	Prueba que fija el contrato del parámetro y controlador leyendo <code>supplierId</code> en lugar de <code>pid</code>	1	P1	—

Tarea	Título	DT	Técnica que aplica	Entregable	SP	Prio.	Depende de
T-09	Caracterizar la autorización de inventario y proveedores	DT-05	Pruebas de caracterización	Matriz rol por verbo sobre /products y /suppliers, con el comportamiento actual de los tres modelos coexistentes	3	P1	T-11
T-10	Unificar la autorización en middleware de ruta	DT-05	Unificación de capa	requireProjectPermission declarado en la ruta para PUT/DELETE de productos y para los endpoints de proveedor; ensureProductInventoryAccess deja de decidir dentro del controlador	3	P1	T-09
T-11	Declarar coverage.include y publicar la línea base real	DT-06	Ajuste de configuración	coverage.include en ambos workspaces y línea base real medida y anotada en el README	3	P2	—
T-12	Recalibrar el trinquete y eliminar los umbrales vacíos	DT-06	Ajuste de configuración	Umbrales globales sobre la base real y los tres globs del módulo de reportes apuntando a archivos que existen en el reporte	2	P2	T-11
T-13	Crear services/http.js	DT-07	Unificación de capa de servicios	Cliente único con URL base, cabeceras de autenticación y de empresa, y tratamiento del 401 en un solo sitio	3	P2	—
T-14	Migrar Inventario y Dashboard a la capa de servicios	DT-07	Unificación de capa de servicios	Ambas vistas sin fetch crudo y sin authHeader local	2	P2	T-13
T-15	Actualizar el lote de parches y versiones menores	DT-08	Actualización de dependencias	ws, vite, nanoid, brace-expansion, socket.io-parser, dompurify, mongoose, qs y concurrently al parche disponible	2	P2	—
T-16	Resolver uploadthing con su cambio semver-mayor	DT-08	Actualización de dependencias	Versión con parche instalada y subida de archivos verificada a mano	2	P2	T-15
T-17	Definir la escala de z-index en theme.css	DT-09	Unificación de tokens	Tokens por capa (fondo, contenido, cabecera, modal, overlay) sustituyendo los 27 valores sueltos	3	P2	—
T-18	Extraer <AppSurface> con capa de contraste	DT-09	Extracción de componente	Componente con scrim aplicado a las vistas que van sobre el fondo animado, con contraste AA verificado	3	P2	T-17
T-19	Endpoint agregado de resúmenes de presupuesto	DT-10	Extracción a endpoint agregado	GET /api/budgets/summary?projectIds= con caché corta, y el dashboard haciendo 2 peticiones en lugar de 1+N	3	P2	—
T-20	Configurar ESLint y Prettier para Vue 3	DT-11	Actualización de herramientas	Configuración en la raíz del monorepo y código existente conforme	2	P3	—
T-21	Añadir lint y auditoría de dependencias al pipeline	DT-11	Actualización de herramientas	Steps lint y npm audit --audit-level=high en ci.yml	1	P3	T-20, T-15
T-22	Validar las URL de salida antes de cualquier fetch	DT-12	Extracción de método	assertPublicHttpUrl que exige esquema HTTP o HTTPS, resuelve el host y rechaza loopback, rangos privados y enlace local; aplicado al probador y al envío de webhooks	3	P1	—
T-23	Limitar los intentos de autenticación	DT-13	Extracción de método	Middleware de límite por IP y por cuenta sobre /api/auth, con respuesta 429 y sin revelar si el usuario existe	3	P1	—

Tarea	Título	DT	Técnica que aplica	Entregable	SP	Prio.	Depende de
T-24	Añadir las cabeceras de seguridad HTTP	DT-14	Actualización de herramientas	helmet en la pila de middlewares con las cinco cabeceras que exige el plan maestro, y el caso SEC2 de su matriz pasando de Pendiente a Cubierto	2	P2	—
T-25	Poner timeout a las llamadas HTTP salientes	DT-15	Extracción de método	fetchWithTimeout compartido por los seis servicios de integración	2	P2	—
T-26	Parametrizar el pool de Postgres y verificar su certificado	DT-15	Ajuste de configuración	max, connectionTimeoutMillis e idleTimeoutMillis explícitos, y rejectUnauthorized en true con la cadena de confianza correspondiente	1	P2	—
T-27	Crear y destruir las vistas del agente dentro de la transacción	DT-16	Extracción de método	withAgentScope que abre la transacción, crea las vistas, ejecuta y las elimina; ninguna vista temporal sobrevive a la petición	3	P2	—
T-28	Diferir el logo 3D y separar las librerías de animación	DT-17	Extracción de componente con carga diferida	KontrolLogo3D y los fondos cargados con defineAsyncComponent, manualChunks para three, ogl, gsap y lenis, y el bundle de entrada por debajo de 500 kB sin comprimir	5	P2	—

Total: 28 tareas, 72 SP. La diferencia con los 60 SP de la sección 2 son las pruebas de caracterización: allí van implícitas dentro del elemento, y aquí se separan en tareas propias porque se mergean antes y de forma independiente.

Si 72 SP no caben en un sprint. Orden de recorte propuesto: T-28 → T-19 → T-17 y T-18 → T-14 → T-20 y T-21. Con ese recorte quedan 21 tareas y 56 SP.

No se recortan, por este orden de razones:

- **T-22 y T-23** (DT-12 y DT-13). Son las dos explotables por cualquier usuario registrado, y T-23 además evita que un anónimo tumbe el servicio justo en el endpoint que SCRUM-28 ya midió saturado en su carga base.
- **T-01 a T-10** (DT-01 a DT-05). P1 con riesgo de datos. Ninguna tiene sentido a medias: la caracterización sin su refactorización no arregla nada, y la refactorización sin su caracterización se hace a ciegas.
- **T-24** (DT-14). Dos puntos de esfuerzo que cierran un caso de la matriz de trazabilidad del plan maestro abierto desde el 04/09/2026.

6. Verificación del criterio de aceptación

Requisito	Estado
Inventario con al menos ocho elementos priorizados	17 elementos (sección 2)
Cada uno con área, descripción, causa, impacto, esfuerzo y prioridad	Cumplido (sección 2)
Cada uno con técnica de refactorización asignada	Cumplido (sección 2, justificado en la sección 4)
Cada uno con momento de ejecución asignado	Cumplido: los 17 elementos se ejecutan en el Sprint 8 (sección 2, columna Sprint)

Requisito	Estado
Tareas creadas en el backlog para el Sprint 8	Desglosadas en la sección 5 (28 tareas). Pendiente crearlas en Jira a partir de ese desglose
La refactorización no se ejecuta en esta entrega	Cumplido: este documento es el único artefacto de la tarea